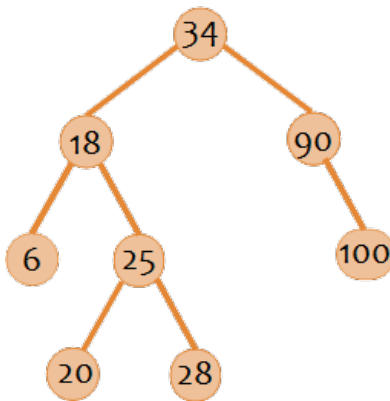


1. Para cada una de las siguientes listas de letras:
 - M, Y, T, E, R
 - R, E, M, Y, T
 - T, Y, M, E, R
 - C, O, R, N, F, L, A, K, E, S
 - a. Dibuja el árbol binario de búsqueda que se construye cuando las letras se insertan en el orden dado.
 - b. Realiza recorridos inorden, preorden y postorden del árbol y muestra la secuencia de letras que resultan en cada caso.
2. Construye un árbol binario de búsqueda para almacenar los datos 12, 8, 7, 16 y 11.
3. Explica gráficamente cómo eliminar los nodos 34, 90 y 6 del siguiente árbol binario de búsqueda:



4. Dado un array de N valores, se puede construir un montículo binario de máximos (*max-heap*) “hundiendo” cada nodo interno hacia su posición.
 - a. A partir del vector [10, 2, 7, 6, 5, 9, 12, 35, 22, 15, 1, 3, 4], crea el árbol binario representado por él, siguiendo la estructura de montículo.
 - b. Este árbol binario es completo, está relleno hacia la izquierda, pero sus nodos no cumplen la relación de orden parcial. Realiza el proceso de **heapify** analizando los nodos internos y “hundiénolos” hacia su posición correcta, comenzando por el nivel n-1 (n=altura del árbol). Ten en cuenta que es un montículo de máximos (*max-heap*), por lo que en la raíz debe resultar el valor máximo.
 - c. Dibuja el montículo después de insertar los valores 16, 20 y 45.
 - d. Dibuja el montículo después de 3 operaciones sucesivas de eliminación del máximo.
5. El algoritmo *heapsort* ordena los elementos de un vector después del proceso anterior, pues podemos ver que no está totalmente ordenado. La operación que se realiza es ir eliminando el elemento máximo (raíz) tal como está en los apuntes e ir almacenándolo en un vector en el que al final estarán ordenados los elementos de mayor a menor si es un *max-heap* y de menor a mayor si es un *min-heap*.
Repite el ejercicio 4 sustituyendo el *max-heap* con un *min-heap* y a continuación aplica el algoritmo *heapsort* para demostrar que los elementos quedan ordenados de menor a mayor.